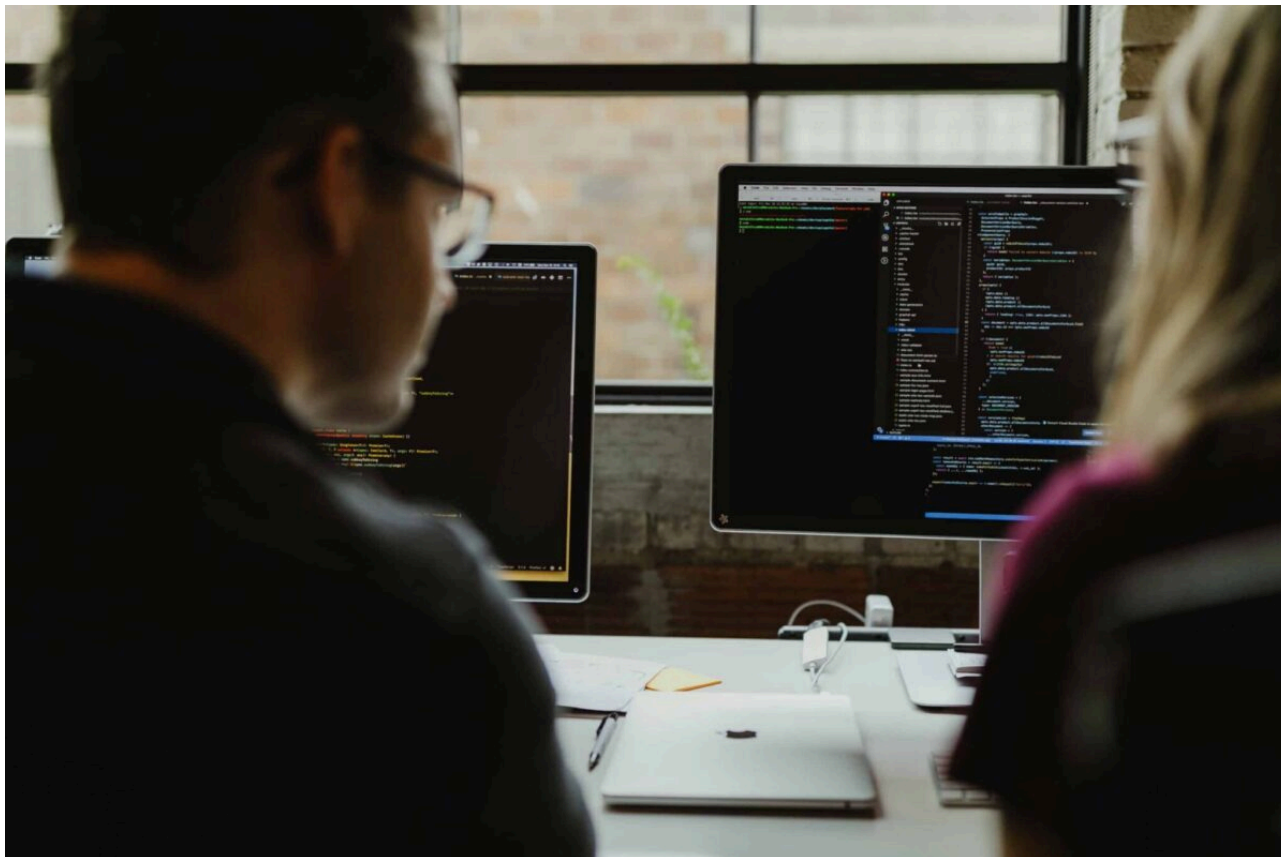


4 терминальных инструмента, которые я рекомендую начинающим разработчикам

СРЕДСТВА РАЗРАБОТКИ АВТОР: НИК ХАЗЕКАМП, 29 ИЮЛЯ 2026 ГОДА



Недавно мы приняли в нашу команду нового талантливого участника **акселератора** (<https://atomicobject.com/accelerator>) Atomic. Он только что окончил университет и имеет ограниченный опыт работы в сфере разработки. В результате есть инструменты, которые я воспринимаю как нечто само собой разумеющееся, но о которых он никогда не слышал. Это в том числе инструменты, которые могут показаться очевидными для опытных разработчиков: **Git**, **mise-en-place** (<https://mise.jdx.dev/>) и такие редакторы, как **Vim**, **VS Code** или **Cursor**, — но для него они либо в новинку, либо используются редко.

Другие инструменты — это приятное дополнение. Это инструменты, которые появились в моей жизни разными путями. Я находил интересные инструменты, просматривая **Lobsters** (<https://lobste.rs/>) и **Hacker News** (<https://news.ycombinator.com/>), по рекомендациям коллег и в процессе парного программирования. Эти инструменты не являются обязательным условием для того, чтобы стать разработчиком. Их стоит освоить, когда вы начнете понимать, какие проблемы они решают.

Вот четыре инструмента, которые стоит попробовать, если они решают проблемы в вашем рабочем процессе:

RELATED POSTS

DEVELOPER TOOLS

([HTTPS://SPIN.ATOMICOBJECT.COM/TOOLS/](https://spin.atomicobject.com/tools/))

How I Rebuilt My Development Workstation for Agentic Work (<https://spin.atomicobject.com/tools/development-workstation/>)

DEVELOPER TOOLS

([HTTPS://SPIN.ATOMICOBJECT.COM/TOOLS/](https://spin.atomicobject.com/tools/))

Ghostty (или какой-нибудь эмулятор терминала)

Встроенный терминал в редакторе удобен на первых порах, но он привязывает вашу работу с командной строкой к конкретному окну редактора. Если вы используете сервер, тестовый наблюдатель и агент кодирования вместе с редактором, выделенный терминал позволит вам работать отдельно. Кроме того, он поможет выработать привычки работы с командной строкой, которые пригодятся вам в разных редакторах и проектах.

Сейчас я использую **Ghostty** (<https://ghostty.org/>), потому что он похож на нативное настольное приложение и включает в себя функции, которые я ожидаю от современного терминала, в том числе вкладки, разделение экрана и темы оформления. **Ghostty** в настоящее время доступен для macOS и Linux. Какие бы терминальные инструменты вы ни выбрали, освоите командную строку. Узнайте, как пользоваться **Vim** (и как из него выйти).

tmux (или другой мультиплексор терминалов)

По мере того как вы все больше работаете в командной строке, количество окон и вкладок растет. Для одного проекта могут потребоваться отдельные терминалы для сервера, средства запуска тестов, агента кодирования и для обычной работы в командной строке. Если вы работаете с несколькими проектами — у меня их больше четырех, — проблема усугубляется.

Когда это начинает казаться неудобным, на помощь приходит мультиплексор. **Ghostty** управляет окнами терминала, а **tmux** — длительными сеансами терминала внутри них.

Лично я использую **tmux** (<https://github.com/tmux/tmux/wiki>). **tmux** существует уже почти 20 лет, и я к нему привык. Он также часто доступен на серверах, к которым мне нужно получить удаленный доступ, что помогает мне использовать одни и те же инструменты в разных средах. Тем не менее я собираюсь вскоре попробовать **Zellij** (<https://zellij.dev/>), основываясь на опыте Патрика (<https://spin.atomicobject.com/zellij-terminal-workspace/>).

Первые два инструмента меняют мой подход к работе с терминалом. Следующие два упрощают понимание конкретных задач, связанных с **Git** и **Docker**.

Lazygit

Интерфейс командной строки **Git** обладает широкими возможностями, но по мере роста проекта становится все сложнее отслеживать общее состояние репозитория. Чем больше веток, участников и параллельных изменений, тем важнее становится иметь возможность с первого взгляда оценить рабочее дерево.

Здесь на помощь приходит **lazygit** (<https://github.com/jesseduffield/lazygit>). **Lazygit** предоставляет пользовательский интерфейс для терминала, или **TUI**, который позволяет просматривать состояние **Git** в вашем проекте и управлять им. **Lazygit** становится особенно полезным, когда `git status`, `git log`, и отдельные команды для работы с ветками больше не дают четкого представления о репозитории. Он не заменяет изучение основ **Git**, но может упростить понимание базового состояния **Git**.

Хотите посмотреть, какие файлы изменились? Оформить отдельные изменения? Сравнить ветки? Разобраться в графе коммитов? Все это можно сделать одним взглядом в **Lazygit**.

Lazydocker

Docker commonly supports local applications, databases, and multi-service environments. Its CLI is powerful, but managing several projects can mean remembering commands and flags just to see what is

Swagger In A Box: NPM, APIs, and
Swagger Codegen
(<https://spin.atomicobject.com/swagger-codegen-npm-package/>)

DEVELOPER TOOLS

([HTTPS://SPIN.ATOMICOBJECT.COM/DEVELOPER-TOOLS/](https://spin.atomicobject.com/developer-tools/))

Here's Why I'm Using Zellij – A
Terminal Workspace
(<https://spin.atomicobject.com/zellij-terminal-workspace/>)

running, follow logs, or stop a container.

lazydocker (<https://github.com/jesseduffield/lazydocker>) provides a terminal interface for inspecting and managing containers, images, volumes, and services in one place. It works with an existing Docker engine; it does not replace one. If you regularly lose track of running containers, struggle to find the relevant logs, or encounter port conflicts, lazydocker may be worth trying.

I often discover that Docker Compose cannot start because another project has already mapped the Postgres port. With lazydocker, I can identify the conflicting container, confirm which project owns it, and stop it in seconds.

4 Terminal Tools

I think about all of the amazing tools I have come across in my career and how much they have improved my quality of life. Once you've found the right one, it often just clicks into place and you forget how painful it was before. This is certainly not an exhaustive list, but these are tools I reach for every day to improve my efficiency and make project work easier to manage.

You do not need to install all four terminal tools at once. Start with the one that addresses a frustration you already encounter. If you've made it this far, I hope at least one of these tools is useful or new to you.

Conversation
